TEMA 1

INTRODUCCIÓN

Desarrollo de Interfaces



IES HENRI MATISSE

IES HENRI MATISSE

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

TEMA 1. INTRODUCCIÓN Y USABILIDAD

Contenido

1. Introducción 3
2. Programación orientada a eventos 3
3. Recomendaciones 5
   3.1. Leyes y estudios 7
      3.1.1. Ley de Jakob 8
      3.1.2. Ley de Postel 8
      3.1.3. Ley de Hick 8
      3.1.4. Ley de Miller 8
      3.1.5. Ley de Fitts 8
      3.1.6. Efecto Von Restorff 10
      3.1.7. Peak-End rule 11
      3.1.8. Umbral de Doherty 12
4. Patrones de diseño 12
   4.1. Patrón Modelo-Vista-Controlador 13
   4.2. Patrón Modelo-Vista-Modelo de Vista (MVVM) 15
5. Estándares de facto 16

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

1. Introducción

En la actualidad prácticamente la totalidad de las aplicaciones existentes interaccionan con el usuario a través de una interfaz gráfica de usuario (GUI) derivada, normalmente, de la interfaz del sistema operativo sobre el que se vaya a ejecutar la aplicación: Windows, Linux, entorno Mac...

La facilidad y la riqueza proporcionada por los entornos de desarrollo para crear formularios y agregar controles: botones, listas, marcos, barras de desplazamiento... permite que una misma aplicación pueda tener resuelta su interfaz de usuario de múltiples maneras según la capacidad artística del diseñador.

Las interfaces deben ser homogéneas y compartir tanto detalles de presentación tan evidentes como el color de texto y fondo de la ventana o el tamaño de la fuente como modos de operación como pueden ser Cortar y Pegar.

Alguna de las cuestiones que se deben de resolver cuando se realiza una interfaz son las siguientes:

¿Cómo organizar la ventana?

¿Qué controles utilizar?

Sincronización entre componentes

2. Programación orientada a eventos

La visión de la programación tradicional cambia, puesto que las aplicaciones pasan de seguir una secuencia a **orientarse en función de los eventos o señales que el**

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

usuario o el sistema proporcionan a la aplicación. Se proporcionan herramientas para el dibujo de la interfaz mediante *controles* o componentes que se ubican en la plantilla como: botones, etiquetas de texto, botones de opción, etc. **Posteriormente se enlazan las acciones asociadas a cada uno de los controles como respuesta a una señal o evento**. Por ejemplo, al apretar el botón de guardar se deberán seguir los pasos adecuados para que el fichero se guarde en el lugar indicado por el usuario.

Un evento es cualquier acción reconocida por un control o formulario. Por ejemplo, el hacer un click con el ratón, apretar una determinada combinación de teclas o simplemente que el sistema ha llegado a una determinada hora. Como consecuencia, el objeto afectado responde a dicho evento ejecutando código en el lugar correspondiente.

Cada uno de los objetos que constituyen la interfaz de la aplicación tiene la posibilidad de responder a un conjunto de eventos ya establecidos (por ejemplo, un clic, un doble clic, la carga del objeto en memoria, etc.). Ahora es el programador el que decide a cuáles va a responder cada objeto y cuál será el código que se ejecutará en cada caso. Este **código constituye el llamado procedimiento de evento para ese objeto y ese evento particular**.

Cuando se inicia la ejecución de la aplicación se carga y se presenta el formulario inicial; a continuación, la aplicación queda en espera de que se produzca un evento. Éste puede ser provocado por el usuario, por el mismo sistema, o incluso por alguna acción del código. Seguidamente se producirá la correspondiente respuesta a ese evento ejecutándose el procedimiento de evento asociado, si existe. Otra vez la aplicación quedará en espera de que se produzca un nuevo evento. De esta forma, **mediante el encadenamiento de eventos, el programa se ejecuta**.

Es necesario considerar el conjunto de sucesos o eventos que se puedan dar cuando el usuario interactúa con nuestra aplicación, puesto que en función de los eventos programados el programa se comportará de una forma u otra. Por ello, debe tener en cuenta un conjunto de hipótesis de partida sobre el estado de la interfaz y de lo que puede ocurrir sobre ella. Además, como ya se ha comentado anteriormente, el mismo código puede generar eventos: por ejemplo, la carga de un formulario en memoria da lugar al evento Load, el cambio del contenido de un cuadro de texto provoca el evento Change, etc.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

3. Recomendaciones

Los usuarios quieren interfaces que cubran sus necesidades y les faciliten la información que necesitan. Quieren tener la sensación de *conducir* el ordenador, no que el ordenador les *conduzca* a ellos, en todo caso que les ayuden.

La principal recomendación a tener en cuenta es **crear la aplicación desde un punto de vista de usuario**, es decir, pensar en el manejo de la aplicación y su funcionalidad para vislumbrar la organización de los elementos o controles en la pantalla.

Para conseguir esta sensación existen las siguientes normas genéricas para la realización de las interfaces gráficas. A continuación, se describen algunas de ellas:

- **Dar el control al usuario**. Definir los modos de interacción de manera que no obligue a que el usuario realice acciones innecesarias y no deseadas. Por ejemplo, si en un procesador de textos se selecciona el corrector ortográfico, el usuario permanecerá en este modo si desea continuar y tendrá la posibilidad de entrar y salir del modo fácilmente.
- **Ocultar al usuario los detalles técnicos**. El usuario no tiene porqué conocer las funciones internas del sistema como la gestión de archivos o la gestión de memoria.
- **Legibilidad**. Para que la IU favorezca la *usabilidad* del sistema de software, la información que se exhiba en ella debe ser fácil de ubicar y leer. Para lograr obtener este resultado se deben tener en cuenta algunas consideraciones como: el texto que aparezca en la IU debería tener un alto contraste, se debe utilizar combinaciones de colores como el texto en negro sobre fondo blanco o amarillo suave. El tamaño de las fuentes tiene que ser lo suficientemente grande como para poder ser leído en monitores estándar. Se debe de resaltar aquello que es importante, como por ejemplo un dato mal introducido o resaltar los números negativos en rojo.
- **Permitir que la interacción del usuario se pueda interrumpir y deshacer**, es decir, que las acciones sean reversibles.
- **Reducir la carga memorística del usuario**, cuanto más tenga que recordar un usuario, más propensa a errores será su interacción con el sistema. Esta es la razón por la que una interfaz de usuario bien diseñada no pondrá a prueba la memoria del usuario.
- **Establecer valores por defecto útiles**. Evita cambios innecesarios si los valores por defecto son los que más se utilizan.
- **Agrupar en paneles los datos que tienen una característica común**. Por ejemplo, si en un formulario se pide información de una persona podemos

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

distinguir entre la información personal y profesional, cada parte estaría separada en paneles distintos.

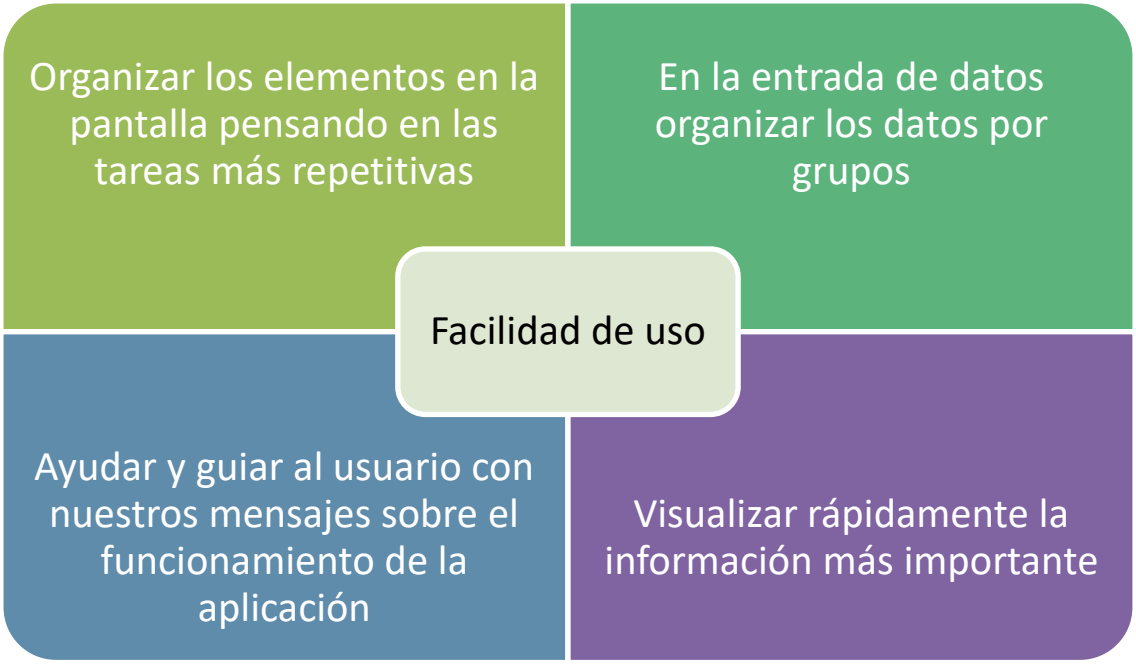
- **Adaptarse a usuarios con distintos niveles de habilidad**. Nuestra aplicación será utilizada por diferentes personas con un grado de conocimiento sobre la misma dispar, por ello deberá adaptarse al grado de conocimiento del usuario. Para usuarios neófitos resulta interesante la utilización de asistentes que nos ayuden a realizar las tareas más comunes, mientras que para los usuarios más expertos podemos implementar atajos de teclado o barras de herramientas que simplifiquen la ejecución de las tareas.
- **Construir una interfaz consistente**: La interfaz deberá presentar la información de forma consistente, esto implica que toda la información visual esté organizada de acuerdo con el diseño estándar que se mantiene en todas las presentaciones de pantallas del entorno gráfico que se haya elegido para la aplicación. Mantener la continuidad dentro de y entre productos. La gente lleva mal el cambio radical de las interfaces ¿Cuánta gente utiliza Windows XP porque dice que no se aclara con Windows 7?
- **El tiempo de respuesta debe de ser pequeño** y en caso de operaciones complejas mostrar algún tipo de progreso para informar al usuario.
- **Ayuda**: disponer de facilidades de ayuda en línea que faciliten el uso del programa. Se trata de una de las tareas que más trabajo llevan pero que el usuario agradece más en un principio.
- **Entrada de datos**: es importante que se minimice el número de acciones que el usuario debe realizar. Además se debe de mantener un orden lógico en la entrada de datos, es decir, que el desplazamiento del **foco** tras completar un campo tenga cierto sentido y orden y no se desplace de forma aleatoria por todo el formulario.
- **Visualización de datos**: utilizar un formato de datos que permita una asimilación rápida de la información y que no varíe de un elemento a otro.
- **Validación de la entrada de datos del usuario**. Resulta conveniente validar muchos de los campos del formulario de introducción para evitar errores no contemplados en el programa. Uno de las principales formas de atacar un programa por parte de un hacker es precisamente ésta, es decir, buscar una introducción de datos que provoquen un desbordamiento de pila y nos permitan acceder al sistema en modo supervisor.
- **Manejo de errores y visualización de mensajes de error**. Es fundamental el uso de excepciones en la programación actual, si se produce un error por algo no contemplado en el programa o por un fallo en el sistema, se deberá

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

capturar esa excepción y facilitar su resolución. **¿Qué imagen da un programa que se cierra al producirse un error?**

- **Gestionar correctamente los mensajes de error, facilitando al usuario su lectura y comprensión**: referencia explícita del problema y sugerencia de soluciones. Se debe de evitar dar información técnica sobre los errores, los mensajes deben de ser claros y apuntar a la resolución del mismo.
- **Logs.** Al usuario no se debe de dar información técnica, sin embargo a nosotros nos resulta de vital importancia tener información sobre el error, ya que de esta manera podremos tener un punto de partida para resolverlo. Por ello resulta de vital importancia en el manejo de las excepciones la utilización de un sistema de logs que guarden la información relevante de una aplicación, similar a un visor de sucesos de Windows.
- **Documentación**: comentar el código y documentar las soluciones a determinados problemas resulta de vital importancia, puesto que cuando se realiza una aplicación nos acordamos de las cosas, pero qué sucede 6 meses después, como dice el refrán *Más vale un lápiz corto que una memoria larga*.

A continuación se muestra una figura que nos ayudará a mejorar nuestras interfaces gráficas



3.1. Leyes y estudios

Existen muchos estudios psicológicos sobre como realizar interfaces y estructurarlas para que el usuario se sienta cómodo al utilizarlas.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

3.1.1. Ley de Jakob

Esta ley habla de la importancia de la **familiaridad** que tiene un usuario con determinadas funciones de la interfaz. El aprendizaje de una nueva interfaz cuesta y se traduce en un consumo energético de nuestro cerebro, es por ello, que las interfaces que son similares a una aplicación con la que nuestro usuario está acostumbrado, resultan más atractivas para ellos porque la curva de aprendizaje es menor. La estructura de la pantalla, los flujos de trabajo, la navegación, la situación de elementos esperados, como la búsqueda, son de vital importancia. Los usuarios tienen la tendencia a esperar diseños similares a los que ya están acostumbrados.

3.1.2. Ley de Postel

Facilita la entrada de datos del usuario. Aunque suponga un trabajo adicional para nosotros, debemos realizar una interfaz guiada y de fácil introducción de la información. Todos hemos sufrido alguna página web o aplicación que para poder introducir la información hemos sufrido falta de claridad o escasa ayuda, haciendo esta tarea difícil y farragosa.

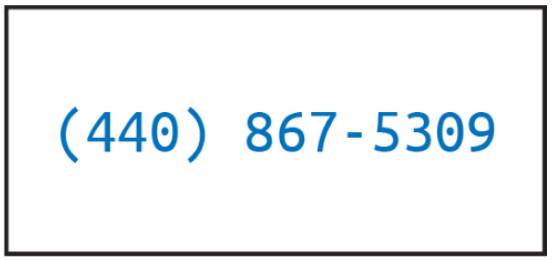
3.1.3. Ley de Hick

El tiempo que se utiliza para tomar una decisión se aumenta con el número de opciones que tenemos. Minimiza las opciones y divide las tareas complejas en pasos más pequeños para disminuir la carga de conciencia cognitiva. Si hay muchos elementos de decisión en la pantalla, la toma de decisiones se vuelve más difícil y lenta.

3.1.4. Ley de Miller

Las personas no tenemos una **gran retención** de elementos en la memoria. La persona promedio puede mantener solo 7 (± 2) elementos en su memoria de trabajo. Organizar el contenido en trozos más pequeños para ayudar a los usuarios a procesar, entender y memorizar fácilmente. Fragmentar la información ayuda a retenerla mejor. Por ejemplo, un número de teléfono.





3.1.5. Ley de Fitts

En la actualidad, y más con la aparición de los dispositivos táctiles, se hace mucho hincapié en el tamaño de las etiquetas o botones que conforman una interfaz GUI. Con la aparición de los dispositivos móviles táctiles y sus reducidas dimensiones esta ley ha cobrado especial importancia.

La ley de Fitts es un **modelo del movimiento humano**, que predice el tiempo necesario para moverse rápidamente desde una posición inicial hasta una zona destino final como una función de la distancia hasta el objetivo y el tamaño de éste.

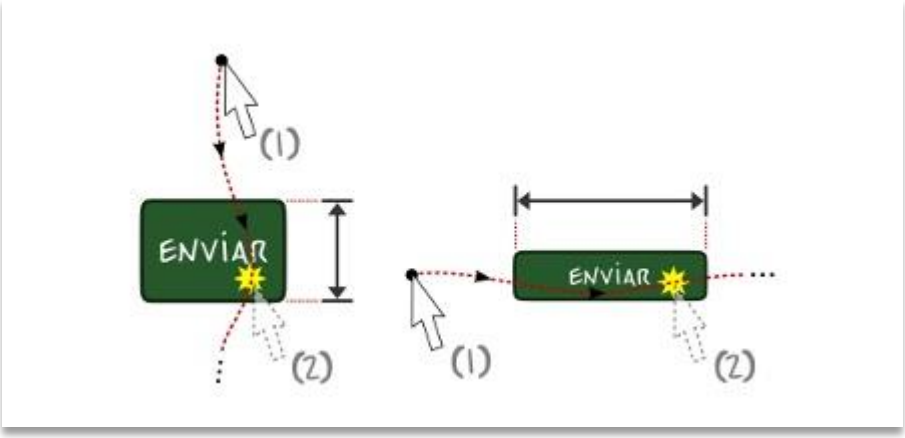
C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Se trata de un modelo matemático que nos ayuda a cuantificar la dificultad que entraña el uso de una interfaz, ya sea en diferentes sectores de la población (jóvenes, ancianos, …) o en condiciones físicas diferentes como pueda ser bajo del agua.

La ley de Fitts realmente nos dice algo obvio *el tiempo necesario para alcanzar un enlace con el puntero es una función de la distancia y del ancho del enlace a lo largo del eje de desplazamiento del cursor*. Más simple aún: si el usuario debe recorrer un espacio en la pantalla con el puntero para hacer clic en un botón, **cuanto más lejos esté el punto de origen y más pequeño sea el botón, más tiempo necesitará**.

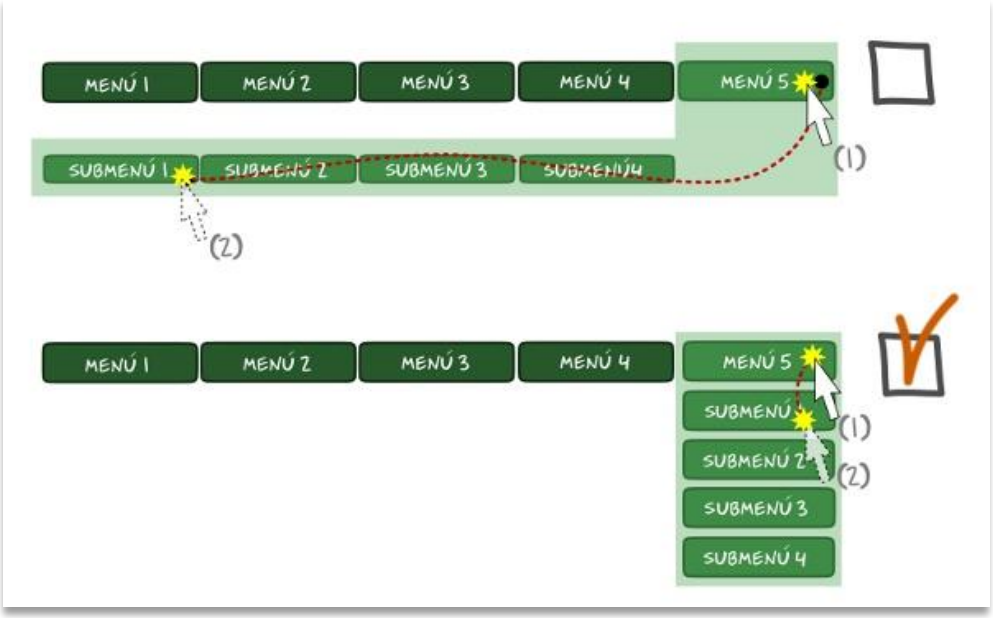
Tan solo grandes compañías, como Apple para sus productos, pueden permitirse valorar sus interfaces con esta ley, pero sí que nos puede ayudar a mejorar nuestras interfaces con algunos consejos:

- Las acciones más habituales o importantes para los usuarios deben estar lo más cerca posible de la posición previa del cursor y tener una superficie clicable mayor. Ejemplo común de mala práctica: al completar el último campo en un proceso de registro, nos encontramos con que el botón *enviar o aceptar* está más lejos de ese punto que otros botones del formulario y su tamaño es exactamente el mismo (¡mal!).
- Si aumentamos el tamaño de los enlaces, debemos tener en cuenta cuál es la dirección de movimiento del cursor más probable cuando éstos se usan, ya que lo que cuenta es el recorrido útil sobre el eje de desplazamiento del cursor. Si el movimiento es predominantemente vertical, por ejemplo, será la altura del botón, y no su ancho, la medida a potenciar.



- En los menús jerárquicos, asegúrate de que las opciones que aparecen quedan cerca de la posición original y que no se cambia la dirección del desplazamiento.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es



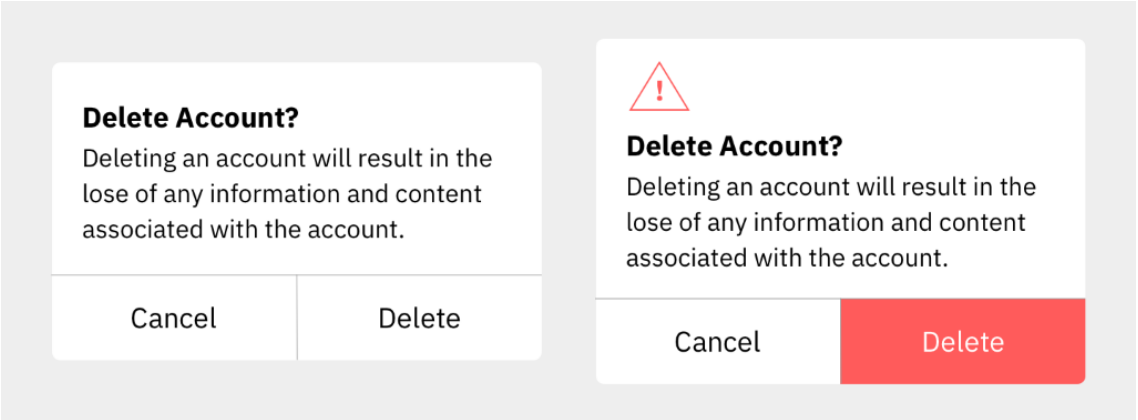
- Los menús popup o contextuales pueden ser usados más rápidamente que los pull-down, al ahorrar desplazamiento el usuario.
- La fórmula nos dice que cada incremento en el tamaño del objeto conlleva una mejora progresivamente menor (un matemático nos diría que esto se debe a la naturaleza logarítmica de la función). La traducción es que hacer enlaces grandes ayuda, pero **hacerlos enormes no aporta ningún beneficio adicional**.
- Quizás no puedas (o no quieras) hacer más grande un objeto, pero posiblemente sí puedes **ubicarlo más cerca**.

Podemos ver una pequeña demostración de la ley de Fitts en el siguiente en la web
http://fww.few.vu.nl/hci/interactive/fitts/

3.1.6. Efecto Von Restorff

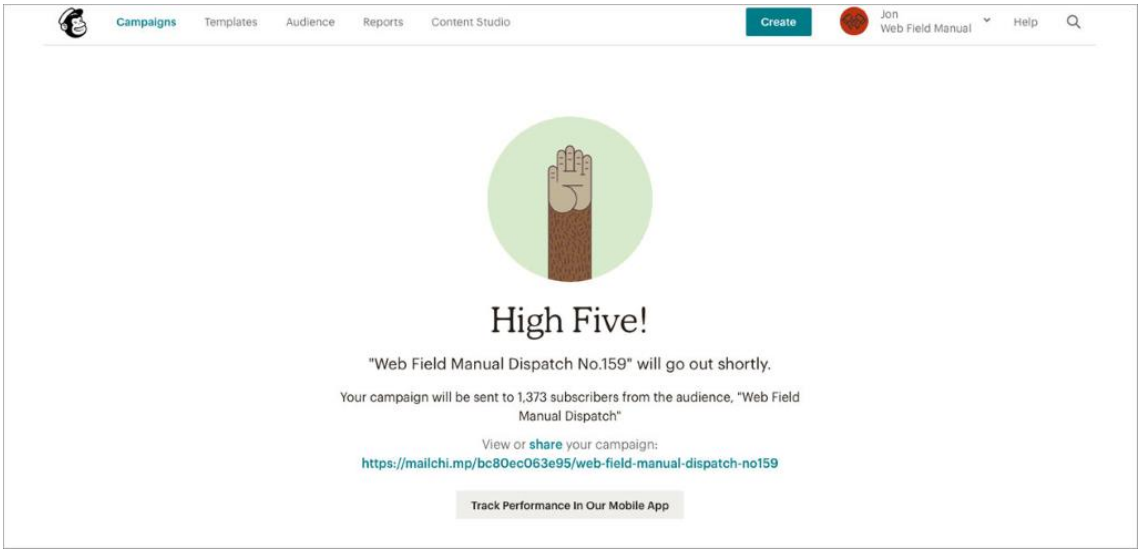
Cuando están presentes múltiples objetos similares, el que **difiere del resto**, es el que tiene más probabilidad de que sea recordado. Haz que la información importante o las acciones clave sean visualmente distintivas.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es



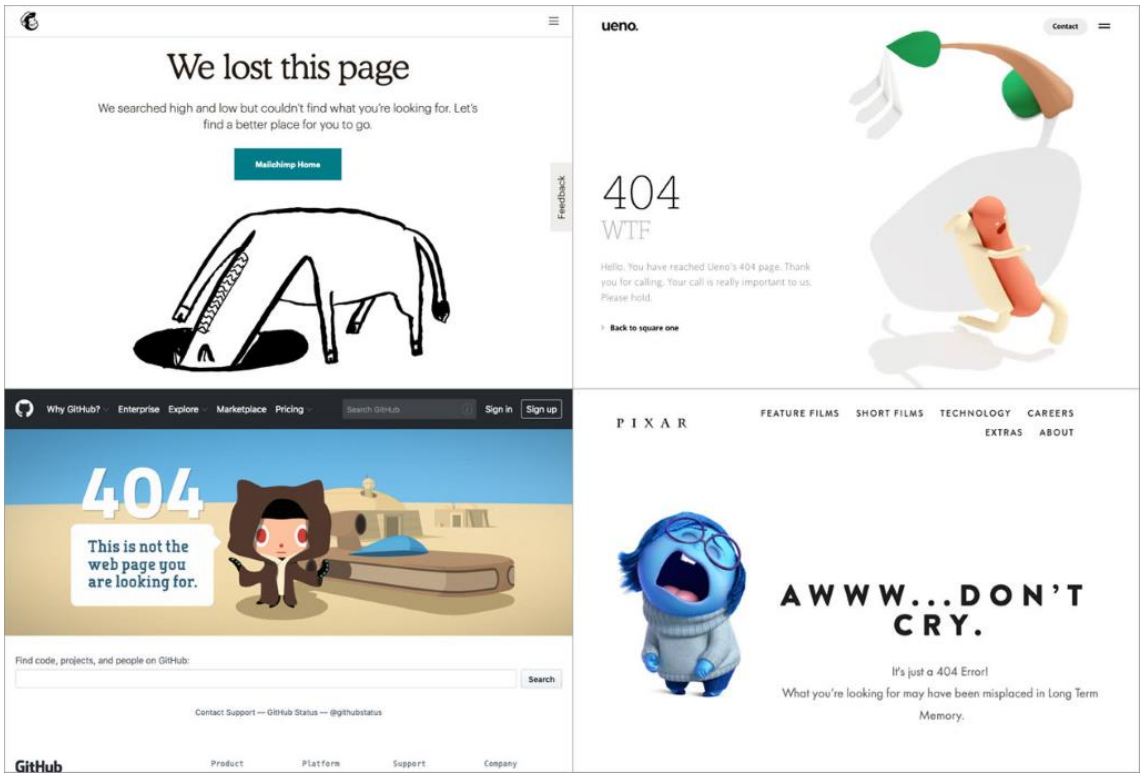
3.1.7. Peak-End rule

Se queda más con **algún momento de apogeo** y con el final más que con la media de utilización. Presta mucha atención a los puntos más intensos y al final de la experiencia del usuario. Mensajes finales con cierto humor y que se recuerden por ser distintos en la forma y no en el fondo.



Intenta suavizar los errores

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es



3.1.8. Umbral de Doherty

La **productividad se dispara** cuando una computadora y sus usuarios interactúan a un ritmo (<400 ms) que asegura que ninguno tiene que esperar al otro. Proporciona retroalimentación dentro de los 400 ms para mantener la atención de los usuarios y aumentar la productividad. Las barras de progreso ayudan a que los tiempos de espera sean tolerables, independientemente de su exactitud

4. Patrones de diseño

Los consejos anteriores son muy importantes y cabe tenerlos muy en cuenta a la hora de realizar las interfaces gráficas, sin embargo, quedan muy abstractos y de primeras no se entienden muy bien.

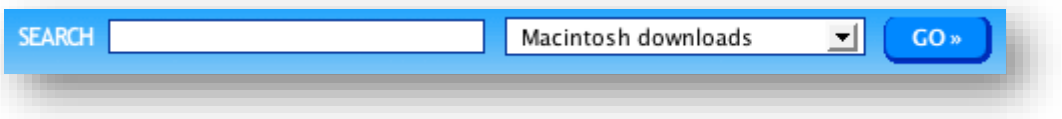
Los patrones de diseño se utilizan en programación en multitud de situaciones y básicamente son ***el esqueleto de las soluciones a problemas comunes en el desarrollo de software***.

En otras palabras, brindan una solución ya probada y documentada a problemas de desarrollo de software que están sujetos a contextos similares.

Existen multitud de patrones para problemas distintos, pero nos centraremos en los llamados patrones GUI que nos ayudarán a resolver de forma efectiva situaciones comunes a la hora de programar aplicaciones gráficas.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

Básicamente este tipo de patrones nos ayudan a visualizar la forma en la que se han resuelto diferentes funcionalidades comunes a muchos programas. Por ejemplo, en nuestra aplicación vamos a realizar una **búsqueda** de registros en una base de datos. Podemos implementarlo de diferentes maneras pero podemos seguir un patrón como el siguiente:



De esta manera podemos mejorar nuestras aplicaciones mediante ejemplos de cosas que ya se han hecho y funcionan, sin contar con la facilidad de uso pues los usuarios ya están acostumbrados a ello.

En el siguiente sitio web podemos encontrar patrones GUI para distintas funcionalidades: http://www.welie.com/patterns/showPattern.php?patternID=search

4.1. Patrón Modelo-Vista-Controlador

El desarrollo de aplicaciones gráficas presenta varias coincidencias, bien sean de escritorio o basadas en web, que nos permiten diseñar un modelo de comportamiento de las mismas de forma genérica.

Es decir, lo que se pretende es tener unas directrices de diseño que nos permitan agilizar la creación de nuestras aplicaciones.

El patrón más extendido para la realización de interfaces gráficas es **modelo-vista-controlador** que pretende la separación de la presentación, de los datos y de los eventos que se producen.

Este modelo consta de tres componentes:

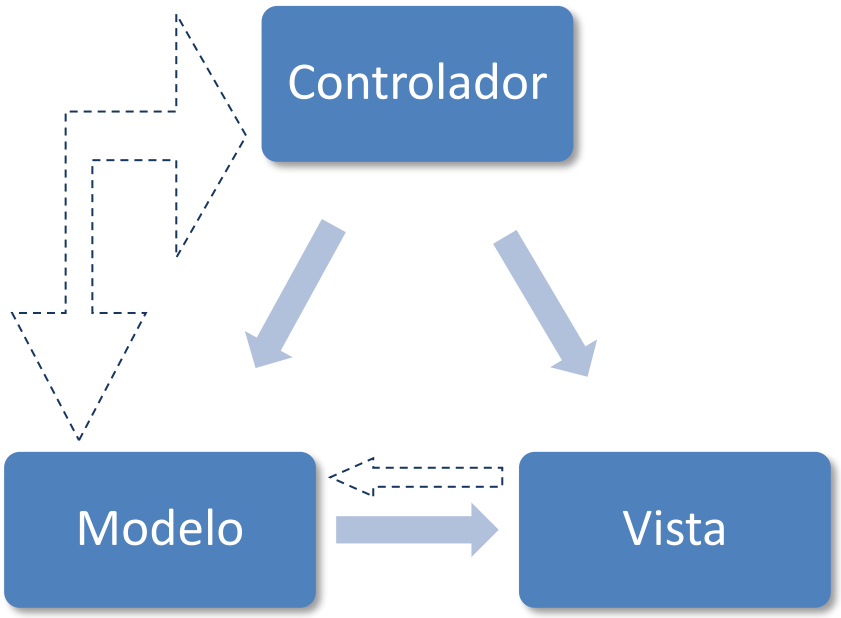
- **Modelo**: es el encargado de almacenar los datos, normalmente reside en memoria, sino todos al menos una parte.
- **Vista**: muestra los datos del modelo, almacenado en memoria, al usuario final.
- **Controlador**: es el encargado de gestionar las relaciones entre el modelo y la vista. Maneja los eventos que se producen en la vista y actualiza el modelo en función de ellos.

Vamos a poner un ejemplo de funcionamiento para comprender mejor el modelo:

Supongamos que hemos desarrollado un programa que contiene una tabla donde visualizamos los datos de una sentencia SQL que hemos realizado. ***El modelo contiene los datos en memoria y la vista los muestra al usuario****. El usuario decide cambiar un dato que considera erróneo, tras hacerlo aprieta el botón de* ***Aceptar****, esto desencadena un evento en el sistema, el controlador lo recibe e indica al modelo que debe de cambiar un dato.*

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

De esta manera funciona el modelo **MVC**, a continuación se muestra una figura que explica:



Ventajas

- Clara separación entre interfaz, lógica de negocio y de presentación, que además provoca parte de las ventajas siguientes.
- Sencillez para crear distintas representaciones de los mismos datos.
- Facilidad para la realización de pruebas unitarias de los componentes.
- Reutilización de los componentes.
- Simplicidad en el mantenimiento de los sistemas.
- Facilidad para desarrollar prototipos rápidos.
- Los desarrollos suelen ser más escalables, es decir, son fácilmente ampliables.

Desventajas

- Tener que ceñirse a una estructura predefinida, lo que a veces puede incrementar la complejidad del sistema. Hay problemas que son más difíciles de resolver respetando el patrón MVC.
- » La curva de aprendizaje para los nuevos desarrolladores se estima mayor que la de modelos más simples como formularios web.
- La distribución de componentes obliga a crear y mantener un mayor número de ficheros.

Pongamos otro ejemplo:

- **Modelo**: un ejemplo de la vida real de un Modelo seria una clase llamada Cliente, la cual tiene las mismas propiedades de una tabla cliente en mi base de datos.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

- **Controlador**: un Controlador seria el Controlador Cliente, generalmente las clases Controladoras llevan el sufijo "Controlador", así que en nuestro caso se llamaría ClientesControlador. El controlador llevaría las acciones que nosotros podemos realizar en un cliente como por ejemplo, agregar, borrar, modificar, agregar orden, etc.
- **Vista**: la Vista es el más fácil de entender, simplemente es nuestra página html. A través de la acción del Controlador especificamos a que vista queremos enviar el resultado de la acción del Controlador. En algunos casos es necesario pasar información a la Vista desde el Controlador, esto se logra fácilmente en el código de la acción.

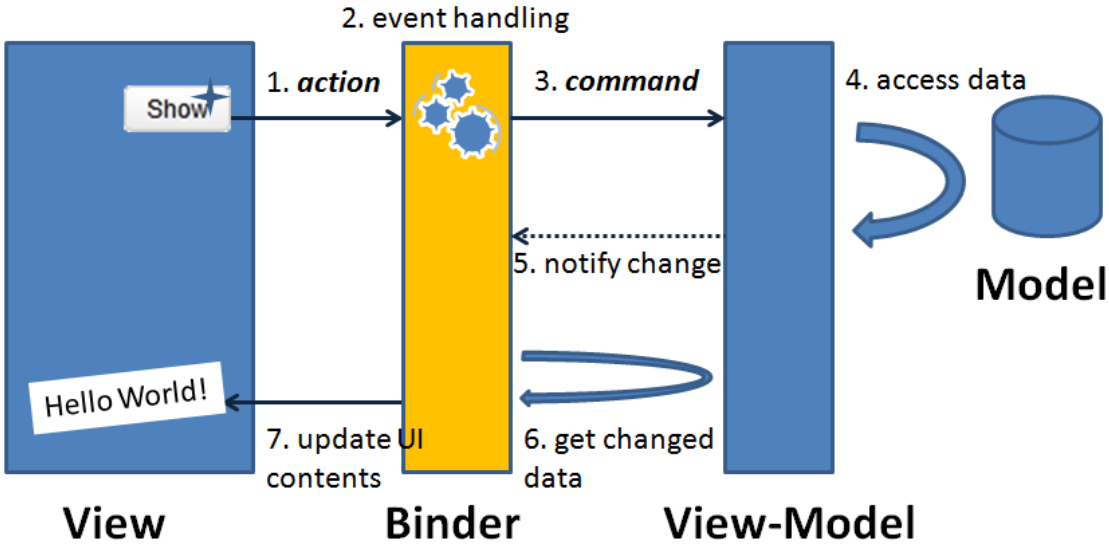
Cace destacar que la mayor parte de las herramientas de desarrollo incorporan en las clases de la vista gran parte o todo el procesamiento de eventos. Con lo que el controlador queda semioculto dentro de la vista.

4.2. Patrón Modelo-Vista-Modelo de Vista (MVVM)

Al igual que el patrón anterior lo que pretende éste es dividir la aplicación en capas lo más independientes posibles para facilitar el mantenimiento y la reusabilidad.

Es muy similar al anterior, de hecho se basa en él, sin embargo añade una capa más para mantener sincronizados los datos entre la vista y el modelo.

En la imagen siguiente se puede ver el esquema de bloques del patrón.



Como más adelante veremos en los ejemplos, está capa realiza el trabajo de sincronización de los datos de forma automática sin necesidad de hacerlo en el controlador.

- **Modelo**: representan los datos que manipula la aplicación. Normalmente sacados de la base de datos.

C/ Enric Valor s/n - 46980 Paterna
Telf.961206280- Fax.961206281
46022622@gva.es
www.ieshenrimatisse.es

- **Vista**: contiene los elementos de la interfaz de usuario. Define la estructura, el diseño y la apariencia de lo que el usuario observa en la pantalla.
- **Modelo de vista**: es la capa intermedia que ofrece los datos a la vista y responde a las interacciones de la misma con eventos que cambian los datos. Para conseguirlo utiliza la técnica del ***binding.***

5. Estándares de facto

Como dice el refrán *cada maestrillo con su librillo* y en la industria informática y no iba a resultar menos. Existen unos estándares de facto que proporcionan diferentes fabricantes de software o hardware para que las aplicaciones sigan los criterios anteriormente descritos.

Por ejemplo, Apple tiene su propia guía de estilo que establece una serie de normas a cumplir para el desarrollo de aplicaciones informáticas.

También resultan interesantes las **normas CUA** desarrolladas por IBM es un estándar de interfaces de usuario para los sistemas operativos y programas informáticos. Fue publicado por primera vez en 1987 como parte de su arquitectura de la aplicación de sistemas. El CUA contiene normas para el funcionamiento de elementos tales como cuadros de diálogo , menús y atajos de teclado que han llegado a ser tan influyente que son implementadas hoy en día por muchos programadores que nunca han leído la CUA.

Algunos ejemplos de combinaciones de teclas que utilizan la mayoría de los programas y sistemas operativos como puede ser invocar la ayuda al apretar F1, realizar una búsqueda al apretar F3 y así un largo etcétera de funcionalidades que se dan como un estándar fueron introducidas por estas normas.

Podemos encontrar una descripción de ellas en la siguiente url normas CUA

Ha habido muchos ejemplos de lo que se llama *Look and Feel*, es decir, los colores utilizados, formas de los elementos, distancia mínima entre ellos, layouts utilizados y el comportamiento de los elementos dinámicos de la interfaz.

Por ejemplo, Apple fue una de las primeras compañías en definir todo este tipo de situaciones.